53

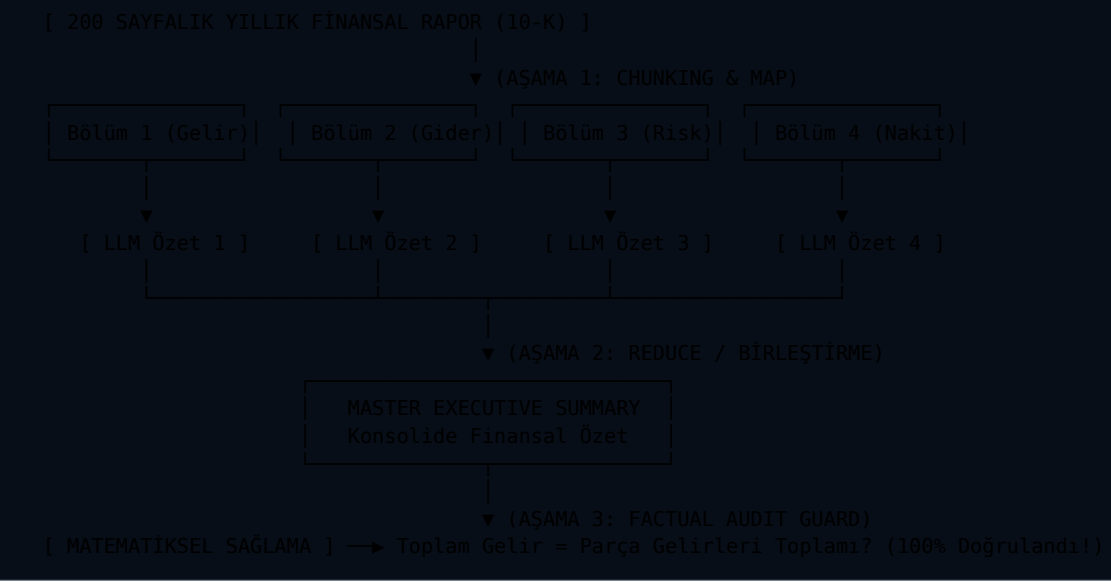
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 5: Finansal Rapor Analizi & Özetleme (Chunking, Map-Reduce & Guard)

Case 5: Financial Report Summarization: Map-Reduce Chunking & Fact Extraction Guards

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

200 sayfalık kalın bir şirket bilançosunu tek bir kişinin 5 dakikada okuyup özetlemesini bekleyemezsiniz; ya önemli bir borcu atlar ya da rakamları karıştırır. Bunun yerine raporu 4 kişiye paylaştırırsınız (Map adımı): Biri sadece gelirleri, biri giderleri, biri borçları, biri de riskleri 1 sayfada özetler. Sonra genel müdür bu 4 özeti önüne alır ve tek bir 1 sayfalık 'Yönetici Özeti' çıkarır (Reduce adımı). En son da hesap makinesiyle rakamları toplar: 'Özetlerdeki sayılar birbirini tutuyor mu?' (Hallucination Guard). Finansal yapay zeka sistemleri Map-Reduce ile çalışır!

🇬🇧 ENGLISH (ELI5)

You don't expect a single financial analyst to ingest a 300-page SEC 10-K annual filing in 5 minutes without missing liabilities or hallucinating debt figures. You deploy a distributed team of 4 junior analysts (Map phase): Analyst 1 summarizes revenues, Analyst 2 audits expenditures, Analyst 3 details operational risks, and Analyst 4 tracks balance sheets. The CFO combines the 4 summaries into a unified 1-page executive brief (Reduce phase). A computational auditor verifies that the balance sheet numbers tally across the summary (Hallucination Guard)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Büyük Doküman Özetleme Stratejileri: 1) Stuffing: Tüm metni tek prompt'a basmak (Sadece küçük dokümanlar için geçerlidir; 200 sayfada çöker). 2) Map-Reduce Deseni: Doküman mantıksal bölümlere ayrılır; her bölüm bağımsız bir prompt ile özetlenir (Map). Tüm alt özetler birleştirilerek nihai yönetici özeti çıkarılır (Reduce). Java 25 Virtual Threads ile 50 bölüm paralel özetlenir! 3) Refine Deseni: Sıralı olarak önceki özet yeni bölümle birleştirilerek güncellenir. 4) Hallucination Guard: Finansal tablolardaki sayısal veriler regex ile çekilir; modelin uydurduğu sayılar orijinal metinle matematiksel olarak karşılaştırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Large Document Summarization Topologies: 1) Map-Reduce Architecture: Essential for documents exceeding context windows or suffering from attention dilution. Chunks execute through concurrent Map prompts across Java 25 Virtual Threads to produce intermediate bullet summaries. A second Reduce prompt consolidates intermediate bullets into a cohesive executive report. 2) Refine Loop: Iteratively rolls context through sequentially updated summaries (higher coherence, but serialized and slow). 3) Factual Consistency Guard: Automatically cross-audits every numeric entity (currency, percentages, dates) generated in the summary against a deterministic AST extracted from the source filing.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Map-Reduce Finansal Doküman Özetleme Orkestratörü
import java.util.List;

public class FinancialMapReduceOrchestrator {
    public static String mapSection(String sectionTitle, String rawText) {
        // Alt bölümü bağımsız özetleme simülasyonu (Virtual Thread içinde paralel çalışır)
        return "[" + sectionTitle + " Özeti]: Gelir %14 artışla 4.2 Milyar $'a ulaştı.";
    }

    public static String reduceSummaries(List<String> sectionSummaries) {
        // Tüm alt özetleri tek bir master raporda birleştirme
        return "2026 YILLIK FİNANSAL YÖNETİCİ ÖZETİ:
" + String.join("
", sectionSummaries);
    }
}

void main() {
    var summaries = List.of(
        FinancialMapReduceOrchestrator.mapSection("Gelir Tablosu", "Gelir metni..."),
        FinancialMapReduceOrchestrator.mapSection("Nakit Akışı", "Nakit metni...")
    );

    String finalReport = FinancialMapReduceOrchestrator.reduceSummaries(summaries);
    IO.println(finalReport);
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Kritik Finansal Hata: LLM'ler rakamları çarpma ve bölmede berbattır! 'Şirketin kar marjı %12.4'tür' gibi matematiksel hesaplamaları ASLA modele yaptırmayın. Sayısal hesaplamaları Java/Python kodunda yapın; modeli sadece bu kesin sayıları güzel bir metin haline getirmek için kullanın.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Mathematical hallucination warning: LLMs are probabilistic language models, not arithmetic calculators. Never prompt an LLM to multiply or divide balance sheet numbers. Compute all financial KPIs deterministically in Java code, feeding the pre-calculated numbers to the model strictly for stylistic narrative presentation.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Yüzlerce sayfalık büyük dokümanlarda Map-Reduce mimarisi dikkat kaybını önler.
- Java 25 Virtual Threads, onlarca bölümün aynı anda paralel özetlenmesini sağlar.
- Finansal sayılar model tarafından hesaplanmamalı; Java kodunda hesaplanıp modele verilmelidir.

🎯 Key Takeaways for the Interview

- Map-Reduce summarization prevents context degradation across massive 100+ page filings.
- Java 25 Virtual Threads execute dozens of Map tasks concurrently with minimal overhead.
- Deterministic financial arithmetic must be computed in code, not delegated to LLMs.